

[48] Faiss: A Library for Efficient Similarity Search and Clustering of Dense Vectors

요약

Faiss는 Facebook AI Research (현재 Meta AI)에서 2017년에 발표한 고성능 벡터 유사성 검색 및 클러스터링 라이브러리이다. C++로 구현되었으며 Python, CUDA 등 다양한 인터페이스를 제공한다. Faiss는 매우 큰 규모의 벡터 데이터(수억 개에서 수십억 개)에 대해 빠르고 효율적인 유사성 검색을 수행할 수 있도록 설계되었다.

Faiss의 핵심은 다양한 인덱싱 방법들을 통합하는 통일된 프레임워크이다. 단순한 선형 검색(Linear)부터 IVF (Inverted File Index)와 PQ (Product Quantization), HNSW (Hierarchical Navigable Small World), LSH (Locality-Sensitive Hashing) 등 다양한 방법들을 지원한다. 사용자는 데이터셋의 크기, 메모리 제약, 정확도 요구사항 등에 따라 적절한 인덱싱 방법을 선택할 수 있다.

Faiss는 또한 GPU 가속을 뛰어나게 지원한다. NVIDIA CUDA를 활용하면 CPU만 사용할 때보다 수십 배 빠른 성능을 얻을 수 있다. 특히 billion-scale의 벡터 검색에서 GPU의 병렬 처리 능력이 매우 유효하다. 이는 실시간으로 수십억 개의 벡터를 검색해야 하는 대규모 웹 검색이나 추천 시스템에서 혁신적이었다.

Faiss는 산업계에서 광범위하게 채택되었으며, 텍스트 검색, 이미지 검색, 추천 시스템 등 다양한 영역에서 사용된다. 또한 오픈소스로 공개되어 연구 커뮤니티에서도 널리 사용되고 있다. Faiss의 등장은 대규모 벡터 검색 시스템의 개발을 크게 단순화했으며, 현대적인 대규모 검색 시스템의 사실상의 표준(de facto standard)이 되었다.

상세분석

48.1 Faiss의 인덱싱 방법들

Faiss는 여러 인덱싱 방법을 제공하며, 각각은 특정한 사용 사례에 최적화되어 있다. 가장 기본적인 방법은 IndexFlatL2이다. 이는 모든 벡터를 그대로 메모리에 저장하고, 쿼리 벡터와의 L2 거리를 정확하게 계산한다. 정확도는 100%이지만, 메모리 사용량이 많고 쿼리 시간이 $O(n \cdot d)$ 이므로 대규모 데이터셋에는 부적합하다.

더 효율적인 방법은 IndexIVFFlat이다. IVF(Inverted File)는 벡터들을 먼저 대략 k 개의 클러스터로 분할한 후, 쿼리와 유사한 클러스터만을 검사하는 방식이다. 각 클러스터는 역파일(inverted list)로 관리되어, 빠른 검색과 메모리 효율성을 모두 제공한다. 검색 시간은 $O((n/k) \cdot d)$ 가 되며, 전체 성능은 k 값에 따라 크게 달라진다.

IndexIVFPQ는 IVF와 PQ를 결합한 방법이다. 각 클러스터 내의 벡터들을 PQ로 압축하므로, 메모리 사용량을 크게 줄일 수 있다. 또한 거리 계산도 PQ의 효율적인 방법을 사용한다. 이는 매우 큰 규모의 벡터 검색에서 가장 널리 사용되는 방법이다.

또 다른 중요한 방법은 HNSW (Hierarchical Navigable Small World) 기반의 인덱싱이다. HNSW는 그래프 기반의 방법으로, 인접한 벡터들을 연결하는 계층 구조를 만든다. 검색은 상위 계층에서 시작하여 하위 계층으로 내려가면서 진행되며, 매우 빠른 검색 속도를 제공한다.

48.2 GPU 가속과 병렬 처리

Faiss의 가장 중요한 혁신 중 하나는 GPU 가속이다. GPU는 대규모 행렬 연산에 매우 효율적이므로, 벡터 간의 거리 계산에 완벽히 적합하다. Faiss는 CUDA를 사용하여 GPU에서의 벡터 연산을 구현했다.

GPU를 사용할 때의 성능 개선은 놀랍다. CPU에서 IndexFlatL2를 사용하여 1억 개의 128차원 벡터를 검색할 때 수십 초가 걸린다면, GPU를 사용하면 수십 밀리초로 단축된다. 이는 단순한 속도 개선을 넘어, 실시간 검색이 가능하도록 만드는 혁신이다.

GPU 가속의 핵심은 배치 처리(batching)이다. 여러 개의 쿼리를 동시에 GPU에 보내면, GPU는 이들을 병렬로 처리할 수 있다. Faiss는 이러한 배치 처리를 자동으로 수행하도록 설계되어 있다.

또한 여러 개의 GPU를 사용하여 처리량(throughput)을 더욱 증가시킬 수 있다. 대규모 데이터센터에서는 여러 GPU 서버를 연결하여, 수십억 개의 벡터를 실시간으로 검색하는 시스템을 구축할 수 있다.

48.3 메모리 최적화 전략

대규모 벡터 검색 시스템에서 메모리는 주요 병목이다. Faiss는 여러 메모리 최적화 기법을 제공한다. 가장 기본적인 것은 PQ를 통한 벡터 압축이다. 128차원 벡터를 32배 압축하면, 1억 개의 벡터도 메모리 수 GB에 저장할 수 있다.

또 다른 최적화 기법은 정량화(Quantization)이다. 벡터를 8비트 또는 16비트 정수로 표현하면, 메모리 사용량을 더욱 줄일 수 있다. Faiss는 여러 종류의 정량화를 지원한다: Scalar Quantization (SQ), Product Quantization (PQ), Binary Quantization 등.

메모리 거래(Memory Trade-off)는 또 다른 중요한 개념이다. 주어진 메모리 제약 하에서, 메모리 사용량을 줄이면 정확도가 낮아지고 쿼리 시간이 길어진다. Faiss는 사용자가 이러한 거래를 명시적으로 제어할 수 있도록 설계되어 있다. 예를 들어, IndexIVFPQ에서 k (클러스터 개수), m (부분공간 개수), $nbits$ (정량화 비트 수) 등을 조정하여 성능을 튜닝할 수 있다.

48.4 인덱스 구성의 유연성

Faiss는 다양한 인덱스 구성 전략을 지원한다. 가장 단순한 것은 단일 인덱스 구성이다. 모든 벡터를 하나의 인덱스에 저장하고, 검색은 이 인덱스에서만 수행한다.

더 복잡한 구성은 인덱스 분할(Index Sharding)이다. 데이터를 여러 개의 샤드로 나누고, 각 샤드에 별도의 인덱스를 구축한다. 검색 시에는 모든 샤드에서 병렬로 검색을 수행하고 결과를 병합한다. 이를 통해 매우 큰 데이터셋도 처리할 수 있다.

또 다른 구성은 다층 인덱싱(Multi-level Indexing)이다. 예를 들어, 먼저 LSH를 사용하여 대략적으로 후보를 선택하고, 그 다음 IVF를 사용하여 더 정밀하게 검색한다. 이러한 다층 구조는 정확도와 성능의 균형을 더 잘 맞출 수 있다.

48.5 클러스터링 기능

Faiss는 단순히 유사성 검색만 제공하는 것이 아니라, 벡터 클러스터링 기능도 제공한다. 이는 데이터의 구조를 이해하고 그룹화하는 데 유용하다.

k-means 클러스터링은 Faiss의 핵심 기능 중 하나이다. 일반적인 k-means 구현과 달리, Faiss의 k-means는 GPU 가속을 지원하므로 매우 빠르다. 수억 개의 벡터를 수십 초 내에 클러스터링할 수 있다.

클러스터링의 결과는 다양하게 활용될 수 있다. 예를 들어, 클러스터 중심을 사용하여 IVF 인덱싱의 클러스터를 초기화할 수 있다. 또한 클러스터링 결과 자체를 분석하여 데이터의 특성을 이해할 수 있다.

48.6 본 논문과의 관계

Exqutor는 유사성 쿼리의 카디널리티 추정을 다룬다. Faiss는 벡터 검색의 가장 중요한 인프라로, Exqutor가 다루는 추정 문제는 Faiss와 같은 벡터 검색 시스템에서 직접 발생한다.

특히, Faiss의 다양한 인덱싱 방법(IVF, PQ, HNSW 등)들은 검색 결과의 크기를 예측하기 어렵게 만든다. 예를 들어, 특정 거리 임계값 이하의 벡터를 검색할 때, 정확히 몇 개의 벡터가 반환될지 예측하는 것은 매우 어렵다. Exqutor의 머신러닝 기반 카디널리티 추정 기법은 Faiss와 같은 현대적인 벡터 검색 시스템에서 정확한 카디널리티 추정을 가능하게 한다.

또한 Faiss의 GPU 가속이 가능해졌다는 사실은 대규모 벡터 검색에서 카디널리티 추정의 중요성을 더욱 강조한다. GPU를 활용한 검색이 매우 빨라지면서, 쿼리 최적화의 오버헤드가 상대적으로 더 중요해진다. 따라서 정확한 카디널리티 추정으로 최적의 실행 계획을 선택하는 것이 전체 성능에 미치는 영향이 더욱 커진다.

추가 제기 문제

1. **동적 인덱스 업데이트:** Faiss의 많은 인덱싱 방법들은 정적 데이터셋을 가정한다. 새로운 벡터가 지속적으로 추가되는 동적 환경에서의 효율적인 인덱스 관리 방법이 제한적이다.
2. **카디널리티 추정의 어려움:** Faiss의 근사 검색 방법들은 검색 결과의 크기를 정확하게 예측하기 어렵게 만든다. IVF+PQ 조합의 경우, 각 단계에서의 근사성이 누적되어 최종 결과 크기 예측이 복잡해진다.

3. **하이퍼파라미터 튜닝:** Faiss는 많은 하이퍼파라미터를 가지고 있으며, 최적의 성능을 위해서는 신중한 튜닝이 필요하다. 특정 데이터셋과 쿼리 패턴에 대한 최적의 설정을 자동으로 결정하는 방법이 부족하다.
4. **메모리 대역폭의 한계:** GPU 가속의 이점에도 불구하고, 메모리 대역폭(memory bandwidth)은 여전히 대규모 검색의 병목이 될 수 있다.